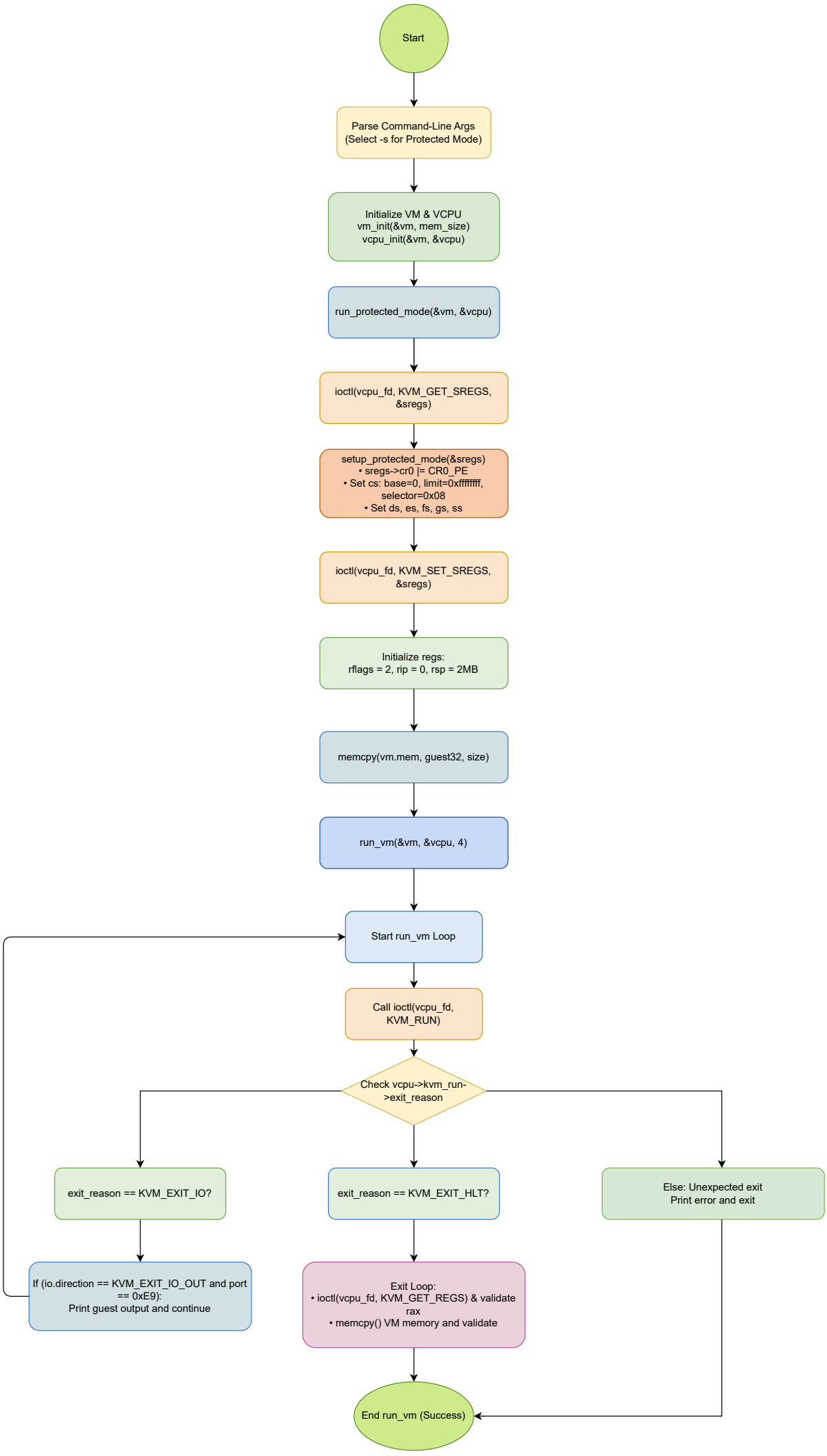
Start
Parse Command-Line Args
(Select -s for Protected Mode)
Initialize VM & VCPU
vm_init(&vm, mem_size)
vcpu_init(&vm, &vcpu)
run_protected_mode(&vm, &vcpu)
ioctl(vcpu_fd, KVM_GET_SREGS, &sregs)
setup_protected_mode(&sregs)
• sregs->cr0 |= CR0_PE
• Set cs: base=0, limit=0xffffffff, selector=0x08
• Set ds, es, fs, gs, ss
ioctl(vcpu_fd, KVM_SET_SREGS, &sregs)
Initialize regs:
rflags = 2, rip = 0, rsp = 2MB
memcpy(vm.mem, guest32, size)
run_vm(&vm, &vcpu, 4)
Start run_vm Loop
Call ioctl(vcpu_fd, KVM_RUN)
Check vcpu->kvm_run->exit_reason
exit_reason == KVM_EXIT_IO?
exit_reason == KVM_EXIT_HLT?
Else: Unexpected exit
Print error and exit
If (io.direction == KVM_EXIT_IO_OUT and port == 0xE9):
Print guest output and continue
Exit Loop:
• ioctl(vcpu_fd, KVM_GET_REGS) & validate rax
• memcpy() VM memory and validate
End run_vm (Success)

(a) Declaring External Constant Arrays

```
extern const unsigned char guest64[], guest64_end[];
```

This line declares two external constant arrays of type unsigned char. These arrays signify the beginning and the end of the 64-bit guest binary code. The actual definitions of these arrays are provided by the linker or another object file. The array guest64 contains the first byte of the guest code, while guest64_end marks its end. By determining the difference between these two symbols, the program can ascertain how many bytes need to be copied into the guest's memory when loading the 64-bit code. This is a common method for embedding binary data (like a compiled guest image) into a host application.

(b) Setting Up Paging and CPU Registers for 64-Bit Long Mode

```
pml4[0] = PDE64_PRESENT | PDE64_RW | PDE64_USER | pdpt_addr;
pdpt[0] = PDE64_PRESENT | PDE64_RW | PDE64_USER | pd_addr;
pd[0] = PDE64_PRESENT | PDE64_RW | PDE64_USER | PDE64_PS;

sregs->cr3 = pml4_addr;
sregs->cr4 = CR4_PAE;
sregs->cr0 = CR0_PE | CR0_MP | CR0_ET | CR0_NE | CR0_WP | CR0_AM | CR0_PG;
sregs->efer = EFER_LME | EFER_LMA;
```

This segment establishes the minimal paging structures and configures control registers required for 64-bit long mode:

Page Table Configuration:

1. **PML4 Table**: The first entry pml4[0] points to the Page Directory Pointer Table (PDPT) at pdpt_addr, with flags ensuring it is present, writable, and accessible in user mode.
2. **PDPT Entry**: The first entry pdpt[0] points to the Page Directory at pd_addr, again marked as present, writable, and user-accessible.
3. **Page Directory Entry**: The first entry pd[0] indicates a large page (typically 2 MB) by using the PDE64_PS flag, and is marked present, writable, and user-accessible.

CPU Register Configuration:

- **sregs->cr3** is set to the physical address of the PML4 table.

- **sregs->cr4** is configured with CR4_PAE to enable Physical Address Extension, a prerequisite for 64-bit mode.
- **sregs->cr0** is set with a combination of flags: CR0_PE (enables protected mode), CR0_PG (enables paging), and other flags (CR0_MP, CR0_ET, CR0_NE, CR0_WP, CR0_AM) that configure various CPU behaviors.
- **sregs->efer** is assigned EFER_LME (Long Mode Enable) and EFER_LMA (Long Mode Active) to finalize the transition to 64-bit long mode.

(c) Allocating Memory for the Guest

```
vm->mem = mmap(NULL, mem_size, PROT_READ | PROT_WRITE,
          MAP_PRIVATE | MAP_ANONYMOUS | MAP_NORESERVE, -1, 0);
madvise(vm->mem, mem_size, MADV_MERGEABLE);
```

This code allocates memory for the guest's physical memory space:

mmap Call:

- Allocates a memory region of size mem_size with read and write permissions.
- MAP_PRIVATE ensures the mapping is private to the process.
- MAP_ANONYMOUS indicates the memory is not backed by any file.
- MAP_NORESERVE tells the kernel not to reserve swap space for this mapping, which is useful for large allocations.

madvise Call:

- The call to madvise with MADV_MERGEABLE suggests to the kernel that this memory can be merged with other identical pages, potentially saving memory when multiple similar regions exist.

Together, these functions set up a large, writable memory area for the guest.

(d) Handling I/O Exit on Port 0xE9

```
case KVM_EXIT_IO:
   if (vcpu->kvm_run->io.direction == KVM_EXIT_IO_OUT &&
      vcpu->kvm_run->io.port == 0xE9) {

      char *p = (char *)vcpu->kvm_run;
      fwrite(p + vcpu->kvm_run->io.data_offset,
          vcpu->kvm_run->io.size, 1, stdout);
```

```
        fflush(stdout);
        continue;
    }
```

This code handles an I/O exit caused by the guest performing an output operation to port 0xE9:

I/O Exit Context:

- When the guest executes an I/O operation, KVM exits and provides details about the operation in the kvm_run structure.
- The condition checks if the exit is due to an output operation (KVM_EXIT_IO_OUT) and if the port is 0xE9.

Data Handling:

- A pointer p is set to the start of the kvm_run structure.
- The data to be output is located at an offset (io.data_offset) from the start of kvm_run, and the number of bytes is specified by io.size.
- fwrite writes these bytes to stdout, effectively relaying the guest's output to the host's standard output.
- fflush(stdout) ensures the output is immediately visible.
- continue restarts the loop, allowing the guest to resume execution.

This mechanism is typically used for capturing debug messages or other output from the guest.

(e) Copying Guest Memory to a Local Variable

```
memcpy(&memval, &vm->mem[0x400], sz);
```

This line copies a section of the guest's memory into a local variable for verification:

Purpose:

- After the guest has executed its code, the host checks the result by reading a value from the guest memory.
- The memory location at offset 0x400 is expected to contain the result of the guest’s computation (e.g., the value 42).

Operation:

- memcpy copies sz bytes from the guest memory (starting at vm->mem[0x400]) into the local variable memval.
- This copied value is then used (in subsequent code) to verify that the guest ran correctly by comparing it against the expected result.